SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems ***COURSE CODE: CSA04***

COURSE OUTCOME COVERED

CO4: *Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.*

Assessment Tool Weightage: 40%

Dr VIMAL V R

QUESTIONS: 5 *Total Marks:25*

Annexure A

CODING CHALLENGE/HACKATHON

Code of Conduct:

I R. Indhu (192524332) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.
R. Indhu

Signature of the student:

Annexure A

CODING CHALLENGE

S.No	Coding Challenge Questions	Marks
1	Write a program to simulate the three file allocation methods—Contiguous, Linked, and Indexed. The program should accept inputs including the total disk size (number of blocks), the file size(number of blocks required),and the allocation method (1forContiguous, 2for Linked, 3 for Indexed). Based on the selected method, the program must allocate disk blocks accordingly, display the allocated block numbers, and handle fragmentation scenarios, particularly when contiguous allocation fails due to insufficient continuous space. The output should show the list of allocated blocks or an appropriate error message if allocation is not possible. Ensure that built-in memory allocation functions are not used, and assume that disk blocks are numbered starting from 0.	5
2	Develop a program to manage free disk space using a bitmap technique. The program should accept inputs including the total number of disk blocks, the initial bitmap (where 0 represents free and 1 represents allocated), and a request for allocation specifying the number of blocks required. The program must identify and allocate free blocks efficiently, update the bitmap after allocation, and also support deallocation of specified blocks with appropriate bitmap updates. The output should display the updated bitmap after allocation and deallocation operations. Ensure that the program efficiently searches for free blocks and properly handles Cases where sufficient space is not available.	5
3	Implement a program that simulates at least three disk scheduling algorithms: FCFS, SSTF, and SCAN. The program should take as input the disk request queue, the initial head position, and the disk size. It must compute the total seek time for each algorithm, display the order in which disk requests are serviced, and compare their performance. The output should include the seek sequence and total seek time for each algorithm, along with identification of the best-Performing algorithm based on minimum seek time.	5
4	Write a program to implement the C-SCAN and LOOK disk scheduling algorithms. The program should accept inputs including the request queue, the initial head position, and the direction of head movement (left or right). It must simulate both algorithms, calculate the total head movement required to service all requests, and compare their efficiency. The output should display the seek sequence, total head movement, and a conclusion indicating which algorithm performs better along with a brief justification. Optionally, the program may include a graphical visualization of head movement.	5
5	Design a program that integrates file allocation strategies with disk scheduling techniques to optimize system performance. The program should accept inputs such as the total number of disk blocks, file size (number of blocks required), disk request queue, initial head position, allocation method (Contiguous or Indexed), and disk scheduling method (FCFS or SSTF). It must allocate disk blocks based on the chosen allocation strategy, simulate disk scheduling to access the allocated blocks, calculate the total seek time, and compare performance across different combinations. The output should display allocated blocks, seek sequence, total seek time, and identify the best combination of allocation and scheduling methods. Ensure the program handles allocation failures, avoids duplicate block allocation, and maintains efficient scheduling.	5

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. File Allocation — Contiguous, Linked, Indexed

Input format

- Total disk blocks
- File size
- Allocation method: 1 = Contiguous, 2 = Linked, 3 = Indexed
- Then enter the bitmap of the disk (0 = free, 1 = allocated)

```
#include <stdio.h>

int main() {
    int n, f, method, disk[100], i, j, count, start = -1;
    printf("Enter disk size: ");
    scanf("%d", &n);
    printf("Enter disk bitmap (0=free, 1=allocated):\n");
    for (i = 0; i < n; i++)
        scanf("%d", &disk[i]);
    printf("Enter file size: ");
    scanf("%d", &f);
    printf("1. Contiguous\n2. Linked\n3. Indexed\n");
    printf("Enter method: ");
    scanf("%d", &method);
    if (method == 1) {
        count = 0;
        for (i = 0; i < n; i++) {
            if (disk[i] == 0)
                count++;
            else
                count = 0;
            if (count == f) {
                start = i - f + 1;
```

```
break;
        }
    }
    if (start == -1) {
        printf("Contiguous allocation failed: insufficient continuous space.\n");
    } else {
        printf("Allocated blocks: ");
        for (i = start; i < start + f; i++) {
            printf("%d ", i);
            disk[i] = 1;
        }
    }
}
else if (method == 2) {
    count = 0;
    printf("Allocated blocks: ");
    for (i = 0; i < n && count < f; i++) {
        if (disk[i] == 0) {
            printf("%d ", i);
            disk[i] = 1;
            count++;
        }
    }
    if (count < f)
        printf("\nLinked allocation failed: insufficient free blocks.");
}
else if (method == 3) {
    count = 0;
    printf("Index block: ");
```

```
        for (i = 0; i < n; i++) {
            if (disk[i] == 0) {
                printf("%d\n", i);
                disk[i] = 1;
                break;
            }
        }
        if (i == n) {
            printf("\nIndexed allocation failed.");
            return 0;
        }
        printf("Allocated blocks: ");
        for (j = 0; j < n && count < f; j++) {
            if (disk[j] == 0) {
                printf("%d ", j);
                disk[j] = 1;
                count++;
            }
        }
        if (count < f)
            printf("\nNot enough blocks available.");
    }
    else
        printf("Invalid method.");
    return 0;
}
```

Sample Input

Enter disk size: 10

Enter disk bitmap (0=free, 1=allocated):

1 0 0 1 0 0 0 1 0 0

Enter file size: 3

1. Contiguous

2. Linked

3. Indexed

Enter method: 1

Sample Output

```
Enter disk size: 10
Enter disk bitmap (0=free, 1=allocated):
1 0 0 1 0 0 0 1 0 0
Enter file size: 3
1. Contiguous
2. Linked
3. Indexed
Enter method: 1
Allocated blocks: 4 5 6
```

2. Bitmap Free-Space Management

This program supports both **allocation and deallocation**.

```
#include <stdio.h>
int main() {
    int n, bitmap[100], req, i, count = 0;
    int d, block;
    printf("Enter number of blocks: ");
    scanf("%d", &n);
    printf("Enter bitmap (0=free, 1=allocated):\n");
    for (i = 0; i < n; i++)
        scanf("%d", &bitmap[i]);
    printf("Enter number of blocks to allocate: ");
    scanf("%d", &req);
    for (i = 0; i < n; i++)
        if (bitmap[i] == 0)
```

```
        count++;
if (count < req) {
    printf("Allocation failed: insufficient free space.\n");
} else {
    count = 0;
     for (i = 0; i < n && count < req; i++) {
        if (bitmap[i] == 0) {
            bitmap[i] = 1;
            count++;
        }
    }
    printf("Bitmap after allocation:\n");
    for (i = 0; i < n; i++)
        printf("%d ", bitmap[i]);
}
printf("\nEnter number of blocks to deallocate: ");
scanf("%d", &d);
printf("Enter block numbers:\n");
for (i = 0; i < d; i++) {
    scanf("%d", &block);
    if (block >= 0 && block < n && bitmap[block] == 1)
        bitmap[block] = 0;
    else
        printf("Invalid or already free block: %d\n", block);
}
printf("Bitmap after deallocation:\n");
for (i = 0; i < n; i++)
    printf("%d ", bitmap[i]);
return 0;
```

}

Sample Input

Enter number of blocks: 8

Enter bitmap (0=free, 1=allocated):

1 0 1 0 0 1 0 0

Enter number of blocks to allocate: 2

Enter number of blocks to deallocate: 1

Enter block numbers:

2

Output

```
Enter number of blocks: 8
Enter bitmap (0=free, 1=allocated):
1 0 1 0 0 1 0 0
Enter number of blocks to allocate: 2
Bitmap after allocation:
1 1 1 1 0 1 0 0
Enter number of blocks to deallocate: 1
Enter block numbers:
2
Bitmap after deallocation:
1 1 0 1 0 1 0 0
```

3. FCFS, SSTF and SCAN Disk Scheduling

```
#include <stdio.h>
#include <stdlib.h>
void fcfs(int a[], int n, int head) {
    int i, total = 0, cur = head;
    printf("\nFCFS Sequence: %d ", head);
    for (i = 0; i < n; i++) {
        total += abs(cur - a[i]);
        cur = a[i];
        printf("-> %d ", cur);
    }
    printf("\nTotal Seek Time = %d\n", total);
```

```
}
void sstf(int a[], int n, int head) {
    int done[100] = {0};
    int i, j, pos, dist, min, total = 0, cur = head;
    printf("\nSSTF Sequence: %d ", head);
    for (i = 0; i < n; i++) {
        min = 99999;
        pos = -1;
        for (j = 0; j < n; j++) {
            if (!done[j]) {
                dist = abs(cur - a[j]);
                if (dist < min) {
                    min = dist;
                    pos = j;
                }
            }
        }
        done[pos] = 1;
        total += min;
        cur = a[pos];
        printf("-> %d ", cur);
    }
    printf("\nTotal Seek Time = %d\n", total);
}
void scan(int a[], int n, int head, int size) {
    int b[100], i, j, temp, total = 0, cur = head;
    for (i = 0; i < n; i++)
        b[i] = a[i];
    for (i = 0; i < n - 1; i++)
```

```
        for (j = i + 1; j < n; j++)
            if (b[i] > b[j]) {
                temp = b[i];
                b[i] = b[j];
                b[j] = temp;
            }
    printf("\nSCAN Sequence: %d ", head);
    for (i = 0; i < n; i++)
        if (b[i] >= head) {
            total += abs(cur - b[i]);
            cur = b[i];
            printf("-> %d ", cur);
        }
    total += abs(cur - (size - 1));
    cur = size - 1;
    printf("-> %d ", cur);
    for (i = n - 1; i >= 0; i--)
        if (b[i] < head) {
            total += abs(cur - b[i]);
            cur = b[i];
            printf("-> %d ", cur);
        }
    printf("\nTotal Seek Time = %d\n", total);
}
int main() {
    int a[100], n, head, size, i;
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request queue:\n");
```

```
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter initial head: ");
    scanf("%d", &head);
    printf("Enter disk size: ");
    scanf("%d", &size);
    fcfs(a, n, head);
    sstf(a, n, head);
    scan(a, n, head, size);
    return 0;
}
```

Sample Input

Enter number of requests: 8

Enter request queue:

98 183 37 122 14 124 65 67

Enter initial head: 53

Enter disk size: 200

```
Enter number of requests: 8
Enter request queue:
98 183 37 122 14 124 65 67
Enter initial head: 53
Enter disk size: 200

FCFS Sequence: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total Seek Time = 640

SSTF Sequence: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total Seek Time = 236

SCAN Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14
Total Seek Time = 331
```

The program displays the **seek sequence and total seek time** for all three algorithms.

Best algorithm: the one having the **minimum total seek time**.

4. C-SCAN and LOOK

```
#include <stdio.h>
#include <stdlib.h>
```

```
void cscan(int a[], int n, int head, int size, int right) {
    int b[100], i, j, temp, cur = head, total = 0;
    for (i = 0; i < n; i++)
        b[i] = a[i];
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (b[i] > b[j]) {
                temp = b[i];
                b[i] = b[j];
                b[j] = temp;
            }
    printf("\nC-SCAN Sequence: %d ", head);
    if (right) {
        for (i = 0; i < n; i++)
            if (b[i] >= head) {
                total += abs(cur - b[i]);
                cur = b[i];
                printf("-> %d ", cur);
            }
        total += abs(cur - (size - 1));
        cur = size - 1;
        total += size - 1;
        cur = 0;
        printf("-> 0 ");
        for (i = 0; i < n; i++)
            if (b[i] < head) {
                total += abs(cur - b[i]);
                cur = b[i];
                printf("-> %d ", cur);
```

```
            }
        } else {
            for (i = n - 1; i >= 0; i--)
                if (b[i] <= head) {
                    total += abs(cur - b[i]);
                    cur = b[i];
                    printf("-> %d ", cur);
                }
            total += cur;
            cur = 0;
            total += size - 1;
            cur = size - 1;
            printf("-> %d ", cur);
            for (i = n - 1; i >= 0; i--)
                if (b[i] > head) {
                    total += abs(cur - b[i]);
                    cur = b[i];
                    printf("-> %d ", cur);
                }
        }
        printf("\nC-SCAN Total Movement = %d\n", total);
}
void look(int a[], int n, int head, int right) {
        int b[100], i, j, temp, cur = head, total = 0;
        for (i = 0; i < n; i++)
            b[i] = a[i];
        for (i = 0; i < n - 1; i++)
            for (j = i + 1; j < n; j++)
                if (b[i] > b[j]) {
```

```
            temp = b[i];
            b[i] = b[j];
            b[j] = temp;
        }
printf("\nLOOK Sequence: %d ", head);
if (right) {
    for (i = 0; i < n; i++)
        if (b[i] >= head) {
            total += abs(cur - b[i]);
            cur = b[i];
            printf("-> %d ", cur);
        }
    for (i = n - 1; i >= 0; i--)
        if (b[i] < head) {
            total += abs(cur - b[i]);
            cur = b[i];
            printf("-> %d ", cur);
        }
} else {
    for (i = n - 1; i >= 0; i--)
        if (b[i] <= head) {
            total += abs(cur - b[i]);
            cur = b[i];
            printf("-> %d ", cur);
        }
    for (i = 0; i < n; i++)
        if (b[i] > head) {
            total += abs(cur - b[i]);
            cur = b[i];
```

```
            printf("-> %d ", cur);
        }
    }
    printf("\nLOOK Total Movement = %d\n", total);
}
int main() {
    int a[100], n, head, size, dir, i;
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request queue:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter initial head: ");
    scanf("%d", &head);
    printf("Enter disk size: ");
    scanf("%d", &size);
    printf("Enter direction (1=Right, 0=Left): ");
    scanf("%d", &dir);
    cscan(a, n, head, size, dir);
    look(a, n, head, dir);
    return 0;
}
```

Sample Input

```
Enter number of requests: 8
Enter request queue:
98 183 37 122 14 124 65 67
Enter initial head: 53
Enter disk size: 200
Enter direction (1=Right, 0=Left): 1
```

The output gives:

```
Enter number of requests: 8
Enter request queue:
98 183 37 122 14 124 65 67
Enter initial head: 53
Enter disk size: 200
Enter direction (1=Right, 0=Left): 1

C-SCAN Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 0 -> 14 -> 37
C-SCAN Total Movement = 382

LOOK Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 37 -> 14
LOOK Total Movement = 299
```

C-SCAN Sequence: ...

C-SCAN Total Movement = ...

LOOK Sequence: ...

LOOK Total Movement = ...

The algorithm with the **smaller total head movement** is more efficient for that test case.

5. Integrated File Allocation + Disk Scheduling

This combines:

- **Contiguous / Indexed allocation**
- **FCFS / SSTF scheduling**
- Allocation of file blocks
- Scheduling of allocated blocks
- Total seek calculation
- Comparison of combinations

```
#include <stdio.h>
#include <stdlib.h>
void fcfs(int a[], int n, int head) {
    int i, total = 0, cur = head;
    printf("\nFCFS Sequence: %d ", head);
    for (i = 0; i < n; i++) {
        total += abs(cur - a[i]);
        cur = a[i];
        printf("-> %d ", cur);
```

```
    }
    printf("\nTotal Seek Time = %d\n", total);
}
void sstf(int a[], int n, int head) {
    int used[100] = {0};
    int i, j, pos, min, dist;
    int cur = head, total = 0;
    printf("\nSSTF Sequence: %d ", head);
    for (i = 0; i < n; i++) {
        min = 99999;
        pos = -1;
        for (j = 0; j < n; j++) {
            if (!used[j]) {
                dist = abs(cur - a[j]);
                if (dist < min) {
                    min = dist;
                    pos = j;
                }
            }
        }
        used[pos] = 1;
        cur = a[pos];
        total += min;
        printf("-> %d ", cur);
    }
    printf("\nTotal Seek Time = %d\n", total);
}
int main() {
    int disk[100], blocks[100];
```

```
int n, f, method, head, schedule;
int i, j, count = 0, start = -1;
printf("Enter total disk blocks: ");
scanf("%d", &n);
printf("Enter disk bitmap (0=free, 1=allocated):\n");
for (i = 0; i < n; i++)
   scanf("%d", &disk[i]);
printf("Enter file size: ");
scanf("%d", &f);
printf("1. Contiguous\n2. Indexed\n");
printf("Enter allocation method: ");
scanf("%d", &method);
if (method == 1) {
   for (i = 0; i < n; i++) {
      if (disk[i] == 0)
         count++;
      else
         count = 0;
      if (count == f) {
         start = i - f + 1;
         break;
      }  }
   if (start == -1) {
      printf("Contiguous allocation failed.\n");
      return 0;
   }
   for (i = 0; i < f; i++)
      blocks[i] = start + i;
}
```

```
else if (method == 2) {
    for (i = 0; i < n && count < f; i++) {
        if (disk[i] == 0) {
            blocks[count] = i;
            disk[i] = 1;
            count++;
        } }
    if (count < f) {
        printf("Indexed allocation failed.\n");
        return 0;
    }   }
else {
    printf("Invalid allocation method.\n");
    return 0;
}
printf("\nAllocated Blocks: ");
for (i = 0; i < f; i++)
    printf("%d ", blocks[i]);
printf("\nEnter initial head position: ");
scanf("%d", &head);
printf("1. FCFS\n2. SSTF\n");
printf("Enter scheduling method: ");
scanf("%d", &schedule);
if (schedule == 1)
    fcfs(blocks, f, head);
else if (schedule == 2)
    sstf(blocks, f, head);
else
    printf("Invalid scheduling method.");
```

```
    return 0;
}
```

Sample Input

Enter total disk blocks: 10

Enter disk bitmap (0=free, 1=allocated):

1 0 0 1 0 0 0 1 0 0

Enter file size: 3

1. Contiguous

2. Indexed

Enter allocation method: 1

Allocated Blocks: 4 5 6

Enter initial head position: 2

1. FCFS

2. SSTF

Enter scheduling method: 2

Sample Output

```
Enter total disk blocks: 10
Enter disk bitmap (0=free, 1=allocated):
1 0 0 1 0 0 0 1 0 0
Enter file size: 3
1. Contiguous
2. Indexed
Enter allocation method: 1

Allocated Blocks: 4 5 6
Enter initial head position: 2
1. FCFS
2. SSTF
Enter scheduling method: 2

SSTF Sequence: 2 -> 4 -> 5 -> 6
Total Seek Time = 4
```

Allocated Blocks: 4 5 6

SSTF Sequence: 2 -> 4 -> 5 -> 6

Total Seek Time = 4